CLARITYLAB

MULTIMODAL AI LEARNING AGENT

Ask any question.
Get a full explanation with text, diagrams, images, and video, all at once.

https://clarity-lab-gamma.vercel.app/

Learning is still text-only

Most AI tools respond like textbooks.

You ask a question. You get paragraphs.

For complex ideas, text alone is not enough.

Recursion needs to be seen.

Photosynthesis needs to be drawn.

Quantum mechanics needs a visual.

Students are left to imagine what should be shown.

CLARITYLAB — A MULTIMODAL AI LEARNING AGENT

One question. One response. Five types of content.

Text explanation : Tailored to your difficulty level

Visual diagram : Generated and rendered as Mermaid.js

AI image : Created by Imagen on Vertex AI

Short video : Generated by Veo, plays instantly

Follow-up questions: Always know where to go next

The agent assembles all of it for you automatically.

TWO-STAGE PIPELINE

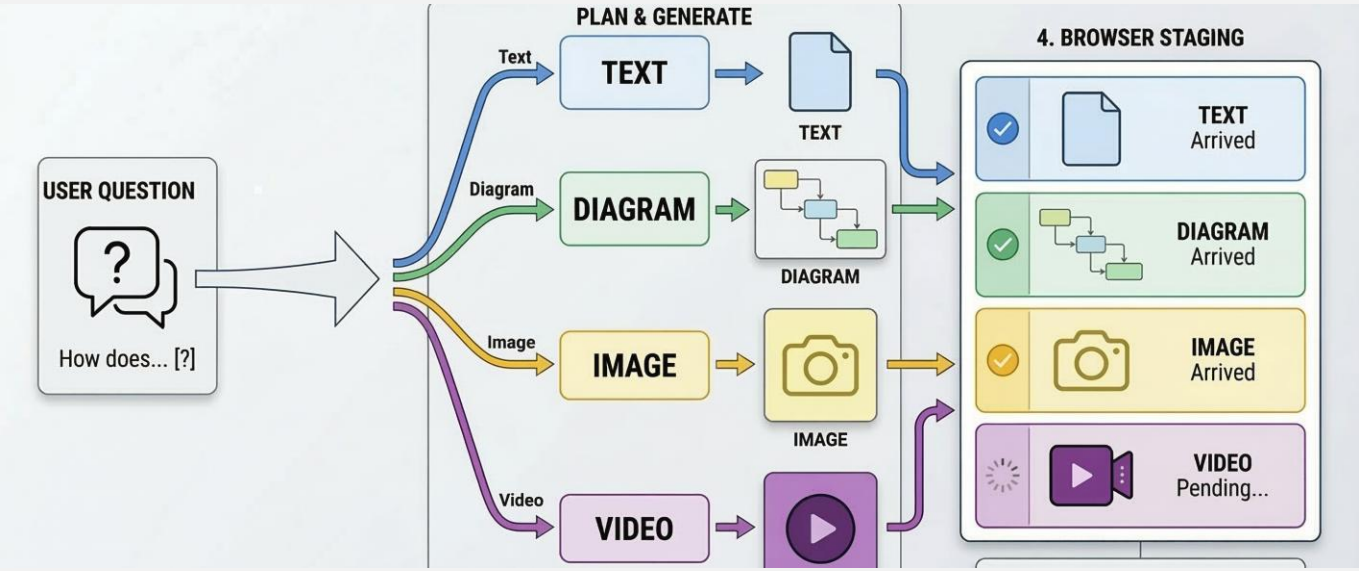
Stage 1: Plan User submits a question POST /api/plan fires ReasoningGenerator analyzes the question Produces 5 optimized prompts (text, diagram, image, video, followups)

Stage 2: Generate in Parallel Frontend fires 5 requests simultaneously /api/generate-explanation /api/generate-diagram /api/generate-image /api/generate-video Each piece renders as it arrives

ARCHITECTURE

Frontend: Next.js 16 / React 19, Vercel

Backend: Python FastAPI, Google Cloud Run



AI AND MODELS

Gemini (text + diagram generation) Imagen on Vertex AI (image generation) Veo (video generation) Google GenAI SDK

BACKEND

Python / FastAPI / Uvicorn Pydantic validation Singleton GeminiClient Dynamic model listing with server-side cache Google Cloud Run

FRONTEND

Next.js 16 / React 19
Mermaid.js (diagram rendering)
react-markdown + KaTeX (math + rich text)
Browser SpeechSynthesis API Vercel

KEY FEATURES

Difficulty control

Beginner / Intermediate / Advanced / Expert / Auto

Asset toggles

Turn diagrams, images, and video on or off per request

Read Aloud

Browser-native TTS narrates the explanation

Dark and light theme

Persistent across sessions, respects system preference

Rich text rendering

Full Markdown, code blocks, mathematical equations

Multimodal image input (backend ready)

Upload images, notes, screenshots for analysis

WHAT WE LEARNED

Multimodal output is a different problem from multimodal input Building coherent multi-type output requires deliberate design

The planning step changes everything Reasoning about what to generate before generating it improves quality across every content type downstream

Parallel async is a UX decision, not just a performance one Progressive rendering makes the tool feel alive Waiting for all five assets feels slow even when it is not

Each content type needs its own prompt language A diagram prompt and a text explanation prompt have almost nothing in common structurally

WHAT IS NEXT

Multimodel Input UI

Backend MultiModalHandler already built Frontend upload interface is the next step

Model Selectio

Let learners choose which Gemini model to use Dynamic model listing endpoint already exists

Persistent Sessions

Cloud Storage for saving generated assets Cloud Run backend deployment

Adaptive Learning

Agent tracks understanding across sessions Adjusts difficulty and framing over time

THANK
YOU